

scripts/build-stats-sample.sh — User Guide

Last verified: 2026-05-14 (1.2.23 cycle) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate) + §11.4.24 (build-resource stats tracking) + Lava §6.AD-debt item 5

Overview

Per-sample resource collector for Lava's build pipeline. Runs in the background during a build, samples memory/CPU%/load every 5s (configurable), then on stop computes per-metric min/max/mean/p95 + appends one summary row to the build registry at `.lava-ci-evidence/build-stats/registry.tsv`. The registry is the single source of truth; the human-readable report is derived by `scripts/build-stats-report.sh`.

The sampler is the operative half of HelixConstitution §11.4.24 — every build exceeding 1 minute **MUST** be sampled so the operator can detect drift in memory / CPU / I/O patterns over time.

Prerequisites

- `bash` ≥ 4 + `awk` + `sort` (POSIX)
- darwin: `vm_stat`, `sysctl`, `ps` (built-in)
- linux: `free`, `ps`, `cut` / `/proc/loadavg`

Usage

Wrap mode (recommended)

```
bash scripts/build-stats-sample.sh wrap "android-debug-1.2.23"
-- ./gradlew :app:assembleDebug
```

Starts the sampler, runs the build command, stops the sampler, appends to the registry. Returns the build's exit code.

Manual start/stop (advanced)

```
# Start; capture sampler PID + tempfile paths.
META=$(bash scripts/build-stats-sample.sh start "my-build"
    "<command-string>")
PID=$(echo "$META" | cut -f1)
SAMPLE=$(echo "$META" | cut -f2)
METAFILE=$(echo "$META" | cut -f3)

# Run your build.
./gradlew :app:assembleDebug
RC=$?

# Stop; append to registry.
bash scripts/build-stats-sample.sh stop "$PID" "$SAMPLE"
    "$METAFILE" "$RC"
```

Inputs

Argument	Required?	Description
wrap <name> -- <cmd...>	yes	One-call: start sampler, run cmd, stop sampler, exit with cmd's rc
start <name> <cmd-string>	yes	Start sampler in background; print <pid>\t<sample-tsv>\t<meta-tsv> to stdout
stop <pid> <sample-tsv> <meta-tsv> [rc]	yes	Stop the sampler, compute aggregates, append registry row
LAVA_STATS_INTERVAL env	no	Sample interval in seconds (default 5)

Outputs

- TSV row appended to .lava-ci-evidence/build-stats/registry.tsv (header auto-created on first run)
- Per-sample tempfiles cleaned up on stop
- One stdout summary line: ✓ build-stats: <name> samples=<N> duration=<s> rc=<code>

Side-effects

- Forks a background process (the sampler) under the calling shell's process group
- Creates `.lava-ci-evidence/build-stats/` if missing (gitignored sample tempfiles; tracked `registry.tsv`)
- Appends to `registry.tsv` (one row per stop)

Heisenberg constraint

Per §11.4.24 the sampler **MUST** stay under 50 MB RSS and 5% CPU. The current implementation invokes `vm_stat / ps / sysctl` once per interval — well within the budget for default `INTERVAL=5s`.

Edge cases

Build too fast (no samples captured)

If the build completes in less than `INTERVAL` seconds, the sampler may produce 0 samples. The stop function logs `△ no samples captured` (build too fast or sampler crashed) and skips the registry append. For short builds, set `LAVA_STATS_INTERVAL=1`.

Sampler crashes mid-build

The sampler runs in a subshell with no error trapping. If it dies (e.g. `vm_stat` fails on an exotic OS), `stop` finds the empty TSV and skips the registry append. The build itself is unaffected.

Concurrent samplers

Multiple builds can sample concurrently — each start produces unique tempfile paths via `$$` (the calling shell's PID). The registry append is atomic-via->> (POSIX guarantees atomic appends $\leq$ `PIPE_BUF` for short lines).

Cross-references

- `HelixConstitution Constitution.md §11.4.24` (the mandate)

- docs/scripts/build-stats-report.sh.md (companion report generator)
- docs/helix-constitution-gates.md (operator-readable gate inventory; this script closes CM-BUILD-RESOURCE-STATS-TRACKER)
- Lava CLAUDE.md §6.AD-debt item 5 (forensic context)